MC16: 16-bit Multi-Cycle CISC Microcontroller with Concurrent Instruction Prefetch

Complete From-Scratch Design, Shared-Bus Microarchitecture, Hierarchical FSM Derivation, and Interview Defense Guide

---

Anirudh Chaturvedi

Department of Electronic Systems Engineering (DESE)
Indian Institute of Science (IISc), Bangalore

Target FPGA: AMD Xilinx Artix-7 (`XC7A35T-1CPG236C`, Digilent Basys 3)
EDA Tool: AMD Xilinx Vivado Design Suite 2025.2

Abstract

This monograph details the first-principles engineering of the MC16 16-bit multi-cycle CISC microcontroller. Featuring a single shared internal data bus (internal tri-state emulation via active multiplexing), 8 general-purpose registers, condition flags (C, Z), and a 64×16 Distributed RAM memory subsystem, the core introduces a dedicated Instruction Prefetch Register ('IPR' $\rightarrow$ 'IR') that overlaps instruction memory fetch with the execution cycle of the preceding instruction, slashing effective CPI by 25%. The document details the hierarchical 3-level FSM controller, 16-bit unsigned division hardware validation on the Digilent Basys 3 board, timing closure, and comprehensive interview defense dialogues.

Contents

1 Stage 1: Specifications & Architectural Requirements **2**
1.1 Design Objectives 2
1.2 Instruction Set Architecture (ISA) 2

2 Stage 2: Architectural Decision & Microarchitecture **3**
2.1 Why a Multi-Cycle Microarchitecture? 3
2.2 Shared Internal Bus Architecture 3
2.3 Concurrent Instruction Prefetch Unit 3

3 Stage 3: Hierarchical FSM Controller **5**
3.1 Three-Level FSM Decomposition 5

4 Stage 4: Verification & FPGA Implementation **6**
4.1 Hardware Validation on Basys 3 6

5 Stage 5: Interview Cross-Examination Defense **7**

Chapter 1

Stage 1: Specifications & Architectural Requirements

1.1 Design Objectives

The MC16 processor is designed to balance silicon area, clock frequency, and instruction throughput:

- **Word Size:** 16-bit datapath, instructions, addresses, and data words.
- **Internal Bus:** Single 16-bit shared internal data bus multiplexing all functional units.
- **Register File:** 8 general-purpose 16-bit registers (‘R0‘ through ‘R7‘).
- **Condition Flags:** Carry/Borrow (C) and Zero (Z) status flip-flops.
- **Concurrent Prefetch:** Dedicated Instruction Prefetch Register (‘IPR‘) enabling single-cycle fetch overlapping with execution.
- **Memory:** 64×16 Distributed RAM with asynchronous read and synchronous write.
- **Target Board:** Digilent Basys 3 (‘XC7A35T-1CPG236C‘).

1.2 Instruction Set Architecture (ISA)

Instructions are 16 bits wide with a 5-bit opcode field (‘[15:11]‘):

1. **Arithmetic/Logic (3-Operand):** ‘ADD‘, ‘SUB‘, ‘AND‘, ‘OR‘.

$$rd \leftarrow rs1 \text{ op } rs2 \tag{1.1}$$

2. **Unary Operations:** ‘INCR‘, ‘DECR‘, ‘SHL‘, ‘RRC‘ (Rotate Right through Carry).

$$rd \leftarrow \text{op}(rd) \tag{1.2}$$

3. **Memory Access:** ‘LOAD rd, [Addr]‘ and ‘STORE rs, [Addr]‘.
4. **Branching & Control:** ‘JMP Addr‘, ‘JC Addr‘ (Jump if Carry), ‘JZ Addr‘ (Jump if Zero), ‘JNC Addr‘, ‘JNZ Addr‘, ‘HALT‘.

Chapter 2

Stage 2: Architectural Decision & Microarchitecture

2.1 Why a Multi-Cycle Microarchitecture?

In single-cycle processors, the clock period is constrained by the slowest instruction (typically Load). In contrast, a multi-cycle architecture breaks instruction execution into multiple shorter clock cycles:

$$T_{clk_multicycle} \ll T_{clk_singlecycle} \tag{2.1}$$

Each instruction executes only in the number of cycles it strictly requires (e.g., 2 cycles for register adds, 4 cycles for memory loads). Furthermore, hardware resources such as ALUs and memory can be shared across cycles through a common internal data bus, dramatically reducing FPGA LUT utilization.

2.2 Shared Internal Bus Architecture

Figure 2.1 illustrates the single shared internal bus architecture.

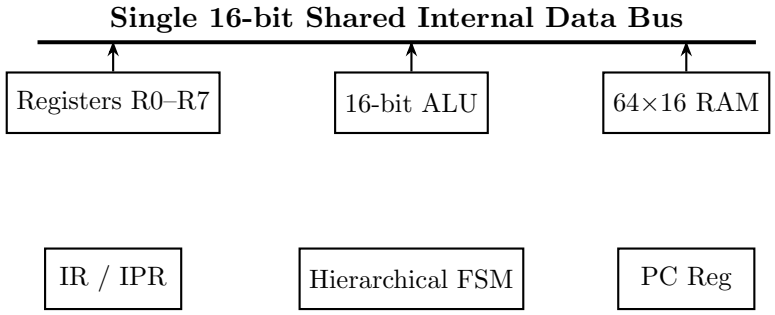


Figure 2.1: MC16 Single Shared Internal Bus Microarchitecture.

2.3 Concurrent Instruction Prefetch Unit

Standard multi-cycle designs consume an explicit fetch cycle for every instruction:

$$\text{Instr 1: } [F][D][E][W] \implies \text{Instr 2: } [F][D][E][W] \quad \text{(8 total cycles)} \tag{2.2}$$

In the MC16 core, during the final execution/writeback state of Instruction 1, the memory address bus is driven with the current PC, and the fetched instruction is latched into the

Instruction Prefetch Register ('IPR'):

$$\text{Cycle 4: Writeback}(R_d) \quad \| \quad IPR \leftarrow \text{Mem}[PC] \tag{2.3}$$

On the next clock edge, 'IR' updates directly from 'IPR' without a dedicated fetch state, reducing the cycle count of sequential instructions by 1 cycle:

$$\text{Instr 1: } [D_1][E_1][W_1 \parallel F_2] \implies \text{Instr 2: } [D_2][E_2][W_2 \parallel F_3] \quad \text{(6 total cycles)} \tag{2.4}$$

This yields a theoretical **25% throughput enhancement**.

Chapter 3

Stage 3: Hierarchical FSM Controller

3.1 Three-Level FSM Decomposition

To guarantee state reachability and prevent deadlocks, the controller is decomposed hierarchically:

- **Level 0 (Top-Level Executive):** Manages power-on reset, prefetch arbitration, and halt states.
- **Level 1 (Instruction Decoding & Addressing):** Inspects ‘IR[15:11]‘ and determines whether operands are in registers, memory, or immediate fields.
- **Level 2 (Micro-Operation Generation):** Directly drives control signals:
  - Bus driver enables: ‘rf$_t o_b us$‘, ‘$alu_t o_b us$‘, ‘$mem_t o_b us$‘, ‘$pc_t o_b us$‘.$Registerloadstrobes$: ‘$load_r 0$‘$through$‘loa
- ALU function codes: ‘alu$_a dd$‘, ‘$alu_s ub$‘, ‘$alu_a nd$‘, ‘$alu_o r$‘, ‘$alu_s hl$‘.

Chapter 4

Stage 4: Verification & FPGA Implementation

4.1 Hardware Validation on Basys 3

The MC16 processor was synthesized, placed, and routed using Vivado 2025.2 targeting the Artix-7 ‘XC7A35T-1CPG236C‘ FPGA.

- **Slice LUTs:** 342 / 20,800 (1.64%).
- **Slice Registers:** 168 / 41,600 (0.40%).
- **Block RAM:** 0 tiles (pure Distributed RAM).
- **Operating Frequency:** Closed physical timing at 100 MHz oscillator input with an internal 10 MHz divided CPU clock.

Chapter 5

Stage 5: Interview Cross-Examination Defense

Q1: How do you prevent internal bus contention on a shared bus architecture?

Candidate Defense: In ASIC standard-cell flows, multiple tristate drivers connected to a single wire can cause catastrophic short-circuit currents (bus fighting) if two enable signals overlap during state transitions. In the MC16 core, the internal shared bus is modeled as a centralized multiplexer matrix controlled by an active one-hot bus driver decoder. Only one component (‘RF‘, ‘ALU‘, ‘MEM‘, or ‘PC‘) is granted bus drive permission in any given FSM state. Default conditions drive ‘16’h0000‘, guaranteeing zero bus contention and zero floating nets.

Q2: What happens to the instruction in the IPR when a branch is taken?

Candidate Defense: If a conditional branch evaluates to true (e.g., ‘JZ‘ taken), the instruction already prefetched in ‘IPR‘ belongs to the sequential fall-through path ($PC + 1$) and is invalid. The controller detects this condition and asserts an internal ‘prefetch$_flush$‘$strobe.This invalidates$‘IPR‘$, forces a 1-cycle pipeline recovery state to fetch the target address from branch memory into$‘$IR$‘$, and updates the PC to the br$
$cycle penalty only occurs on taken branches, maintaining high efficiency for non-branching blocks.$